

1 - UVM TEST & MESSAGES

Learning objectives

After completing this lab, you should be able to

- Create a simple UVM test
- Embed report messages
- Compile the test
- Run the simulation and observe results
- Add environment to test
- Compile and run simulation to observe results

Lab Duration:
45 Minutes

Getting Started

UVM consists of a set of coding guidelines with a set of base classes and macros. The set of base classes and macros assist you in developing testbenches that are consistent in look and feel. The set of coding guidelines enables you to develop testbench components which are robust and highly re-usable. As a result, you will spend less time modifying, maintaining the verification infrastructure and more time verifying your designs.

In this first lab, you will start the process of build a UVM verification environment using the UVM base classes and macros following the UVM coding guidelines.

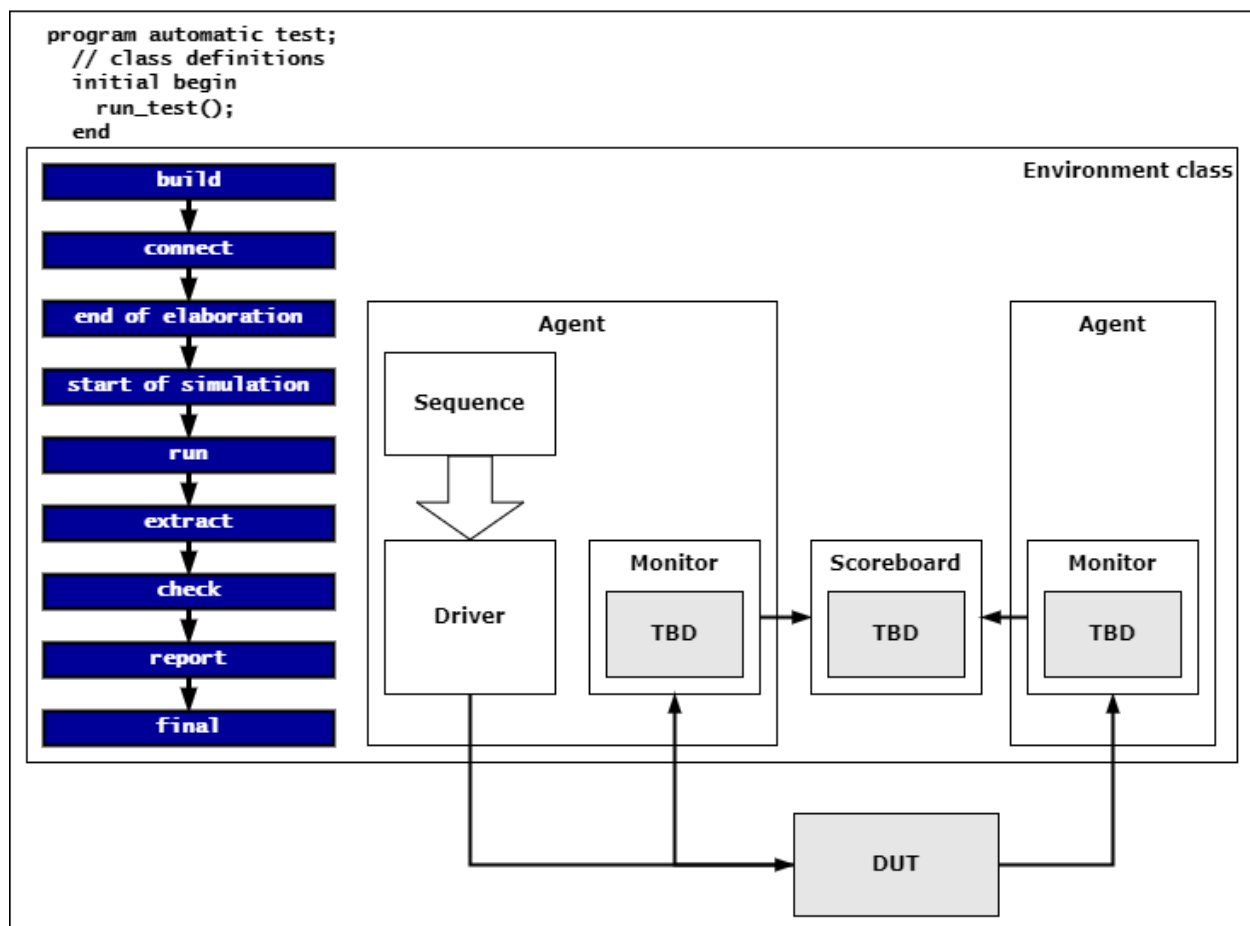


Figure 1. Lab 1 Testbench Architecture

After you log in, you will see three directories: **labs**, **solutions** and **rtl**.

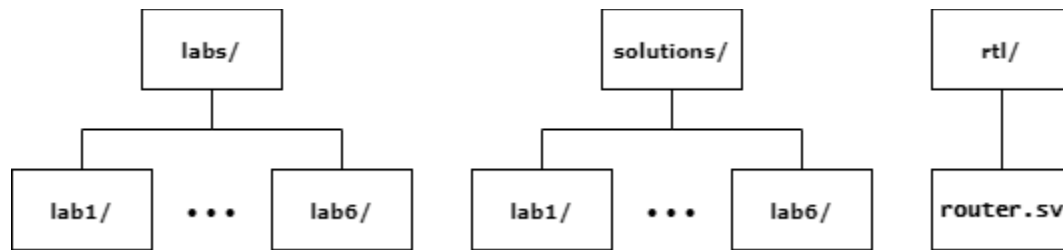


Figure 2. Lab Directory Structure

For each individual lab, you will work in the specified lab directory. Should you have a question during the lab and want to know what the potential solution is, you can flip through to end of the lab to find solution code.

The work flow for this lab is illustrated as follows.

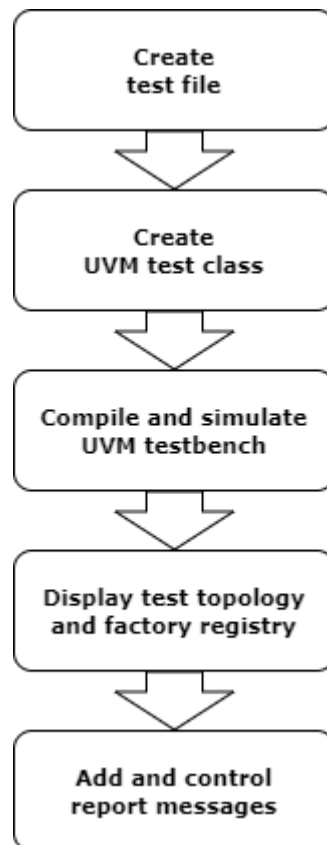


Figure 3. Lab 1 Flow Diagram

DSIM first steps

1. After unzipping the labs, open your VSC and choose “*Open Folder...*”. Find the **ces_uvm-1.2_2021.09** folder that you previously created unzipped and click “*Select Folder*”.

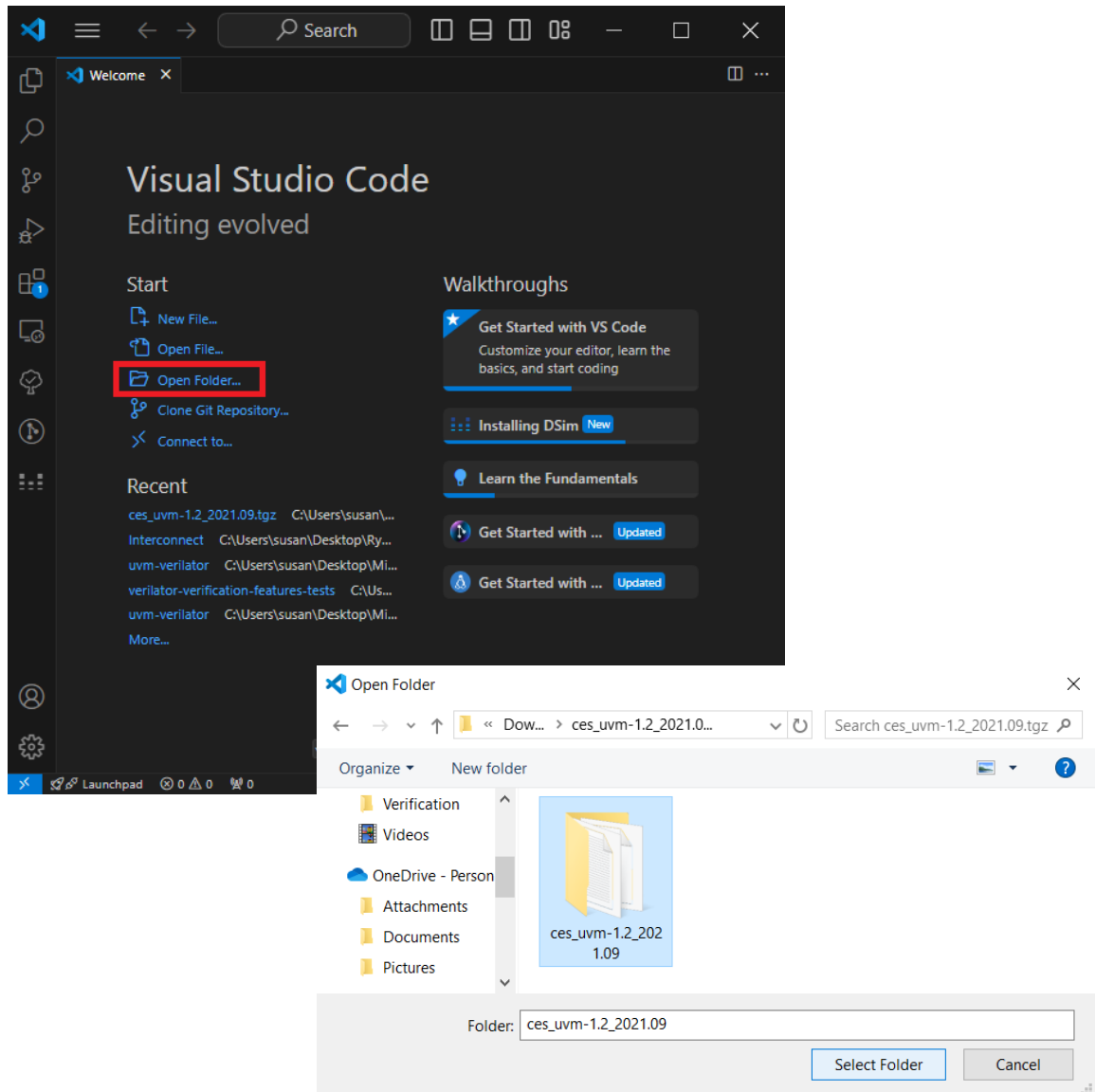


Figure 4. Open working folder.

2. Move to the Explorer tab where the folder should have been uploaded.
3. Inside the **ces_uvm-1.2_2021.09** folder create a new folder: **dsim_filelist**
4. Inside **dsim_filelist** folder, create the files **rtl_filelist** and **verif_filelist**
5. Open **rtl_filelist**, add the next code, and save the file.

```

..\rtl\ral\host_io.sv
..\rtl\reset_io.sv
..\rtl\router_io.sv
..\rtl\router_test_top.sv
..\rtl\router.sv

```

6. Open **verif_filelist**, add the next code, and save the file.

```
rtl\router_test_top.sv
```

7. Right click the **ces_uvm-1.2_2021.09** folder and choose the “*DSim Desktop: Create Project*”.

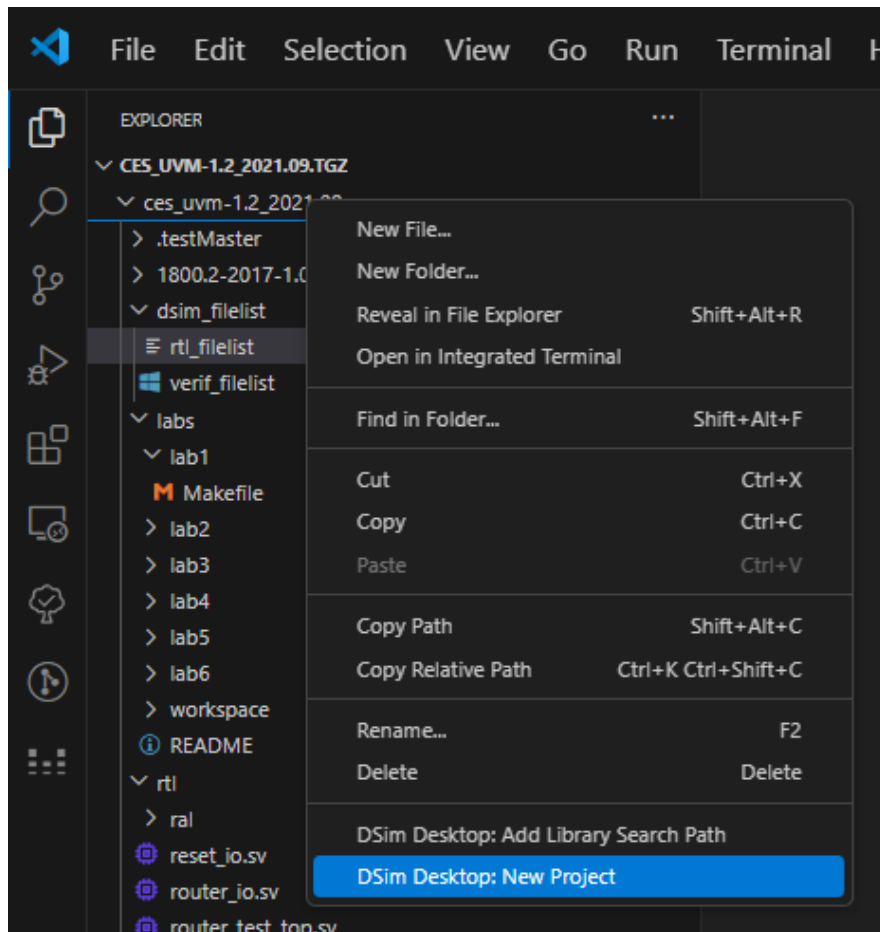


Figure 5. Create a New Project

8. Configure your new project as is shown in figure 7, with project name as **my_labs_sim** and save it. This will generate the file **my_labs_sim.dpf** in the main directory.

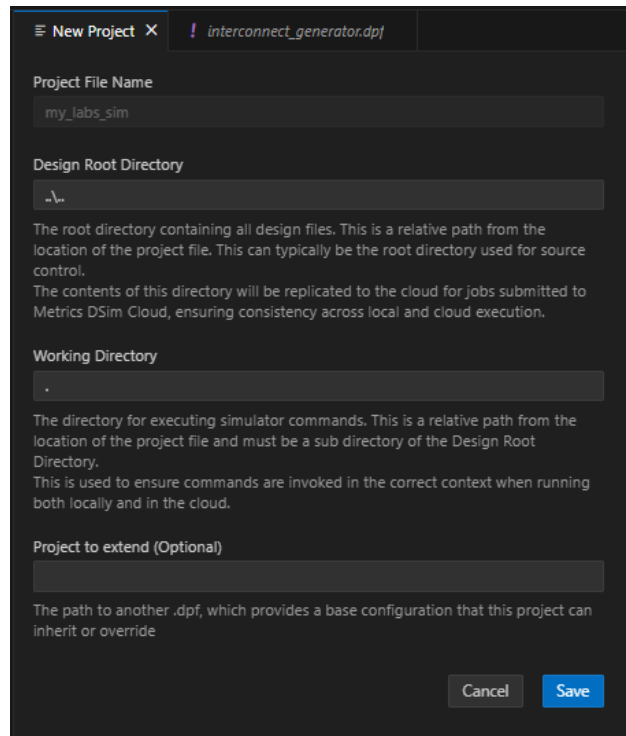


Figure 7. Configuring the project.

9. Right click the **rtl_filelist** file and choose “*DSim Desktop: Configure File*”.

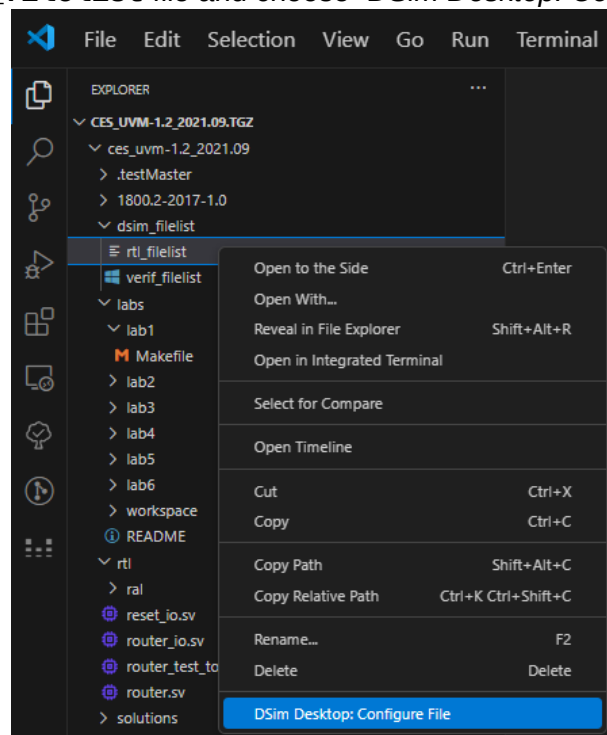


Figure 8. Configure file

10. Fill the configuration tab as shown in Figure 9 for the **rtl_filelist** file, and save it.

Configure File

File Path

dsim_filelist\rtl_filelist

Library Name (Optional)

work

Choose a Language

Verilog / SystemVerilog

☒ Configure as file list

File list contents are relative to

☒ File list file

☐ Work directory

Options (Optional)

+acc+b

Compilation Order (Optional)

0

If the order is not provided, the file will be placed at the end of the list.

Cancel Save

Figure 9. Configuration for rtl_filelist

11. Right click the **verif_filelist** file and choose “*DSim Desktop: Configure File*”.
12. Fill the configuration tab as shown in Figure 10 for the **verif_filelist** file, and save it.

≡ Configure File X

File Path

dsim_filelist\verif_filelist

Library Name (Optional)

work

Choose a Language

Verilog / SystemVerilog

☒ Configure as file list

File list contents are relative to

☐ File list file☒ Work directory

Options (Optional)

-uvm 1.2 +acc+b

Compilation Order (Optional)

1

If the order is not provided, the file will be placed at the end of the list.

Cancel

Save

Figure 10. Configuration for verif_filelist

13. Click the DSim Desktop view and expand the libraries in the **LIBRARY CONFIGURATION**.

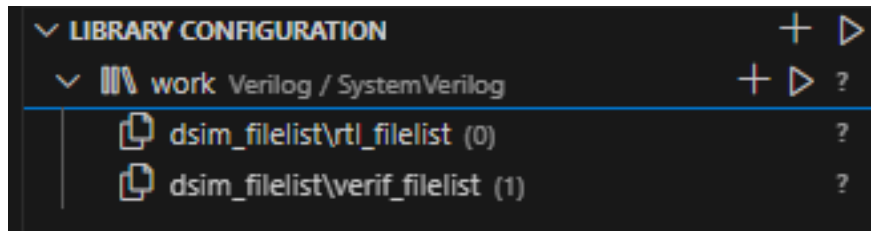


Figure 11. Unknown compilation status.

The file list and file appear in their respective libraries. Note that there are different icons for file lists and files. The ? beside the libraries and files indicate that their Compilation Status is Unknown, ie. they have not been compiled yet.

14. Click the '*Compile Project*' play button under the library configuration to compile all your projects. If you want to only compile one library you can only click the play button for the specific library.

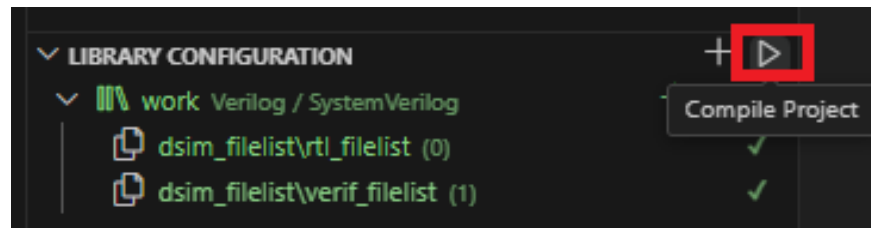


Figure 12. Successful compilation.

OPTIONAL TASK FOR RUNNING DUMMY TEST

15. Hover over the **SIMULATION CONFIGURATION** drop-down menu and click the + button.

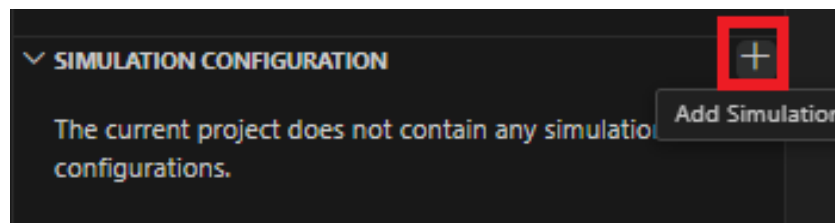


Figure 13. Add a simulation configuration.

16. Fill the configuration for the simulation as shown in figure 14.

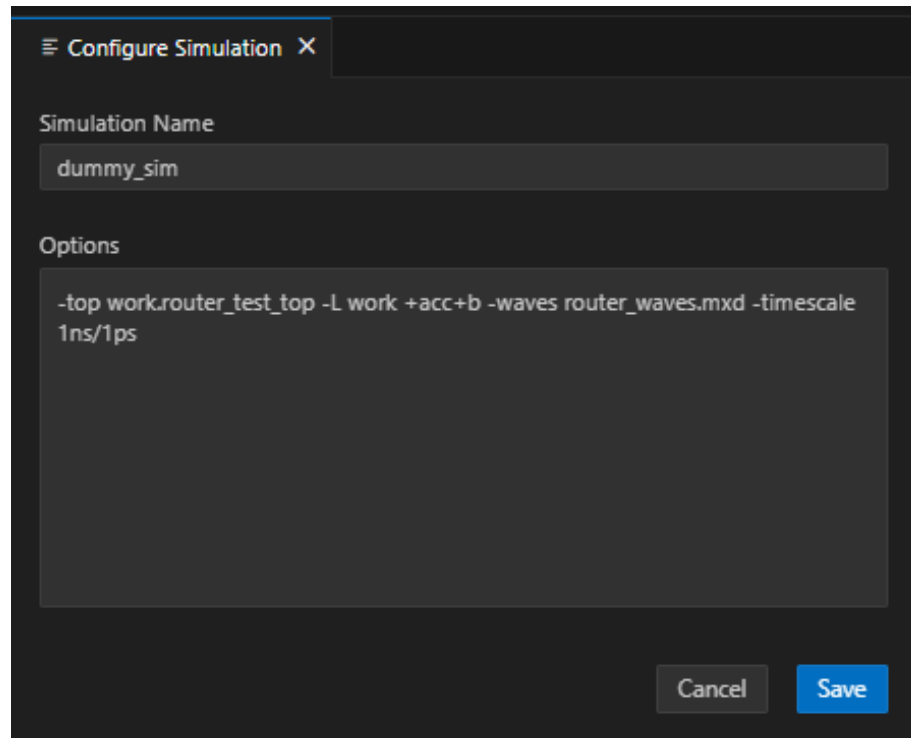


Figure 14. Dummy configuration for sim.

17. Open the file **router_test_top.sv** from the folder rtl.
18. Comment line 11 and add the next code, so that you have a module as shown in figure 15.

```
initial begin
    #100;
    $finish;
end
```

```

1  module router_test_top;
2      parameter simulation_cycle = 100 ;
3      bit SystemClock;
4
5      reset_io reset_if(SystemClock);
6      router_io router_if(SystemClock);
7      host_io host_if(SystemClock);
8      router dut(.clk(SystemClock), .reset_n(reset_if.reset_n), .io(router_if), .host(host_if));
9
10     initial begin
11         // $fsdbDumpvars;
12         forever #(simulation_cycle/2) SystemClock = ~SystemClock ;
13     end
14
15     initial begin
16         #100;
17         $finish;
18     end
19 endmodule

```

Figure 15. Modified router_test_top.

19. Recompile, as shown in step 14.
20. After the successful compilation of your design, run the test by clicking the 'Run' button beside dummy_sim in the **SIMULATION CONFIGURATION** from the DSIM Desktop.

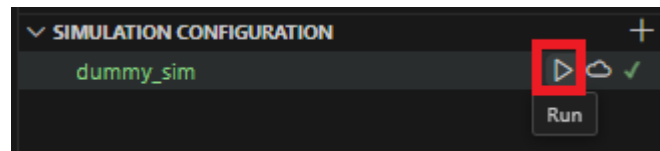


Figure 16. Running dummy test.

If everything is ok, VSC will show the message from the figure 17.

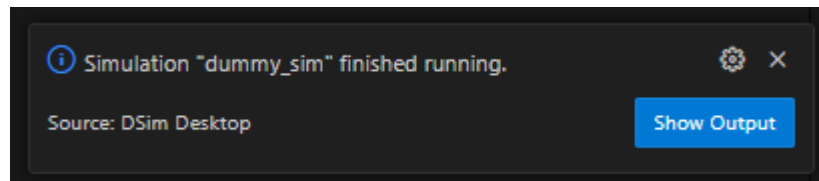


Figure 17. Successful simulation.

Building a UVM Testbench

Task 1. Create a Simple Test

For this first task you will create a simple test and a program block to execute this simple test. Use the lecture material as your reference.

1. Install the WSL extension in VSC. Then, open a new WSL terminal tab.

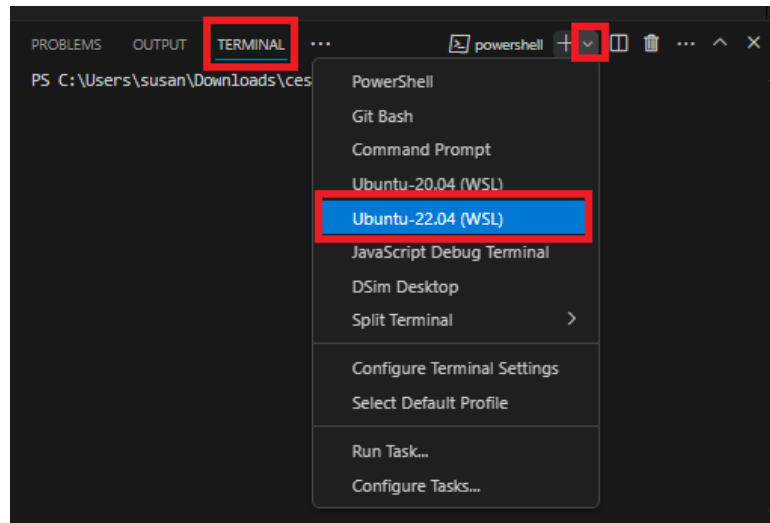


Figure 18. New WSL tab.

Go into **lab1** directory: **ces_uvm-1.2_2021.09/labs/lab1** directory, then use the next command:

> make create

The project will be built. No need to create new files. All the required empty files will be added to the project. All the linux commands can be run in a WSL terminal in VSC.

Every test in Verilog has an entry point. This is done in an **initial** block.

The **initial** block can be coded inside either a **program** or a **module**. Executing testbench code in **program** block will ensure better protection against race conditions than executing the code in **module**. For this reason, in the labs, we are going to run the testbench code in a **program**.

Inside the **program** block, the **import uvm_pkg::*;** statement enables access to all the UVM features. The three most important elements of the **uvm_pkg** are:

- 1- the UVM execution manager (accessed via **uvm_root::get()**)
- 2- the UVM factory (accessed via **uvm_factory::get()**)
- 3- the UVM configuration database (**uvm_config_db**)

To start UVM execution, you must start the UVM execution manager by calling the **run_test()** method in an **initial** block.

2. In **lab1** folder, open the **test.sv** file.
3. Enter the following test code

```

program automatic test;
  import uvm_pkg::*;

  initial begin
    $timeformat(-9, 1, "ns", 10); // Format Verilog time
    run_test();
  end

endprogram

```

4. Open **verif_filelist** to add the next code line and save the file.

```
labs\lab1\test.sv
```

Hover over the **SIMULATION CONFIGURATION** drop-down menu and click the + button.

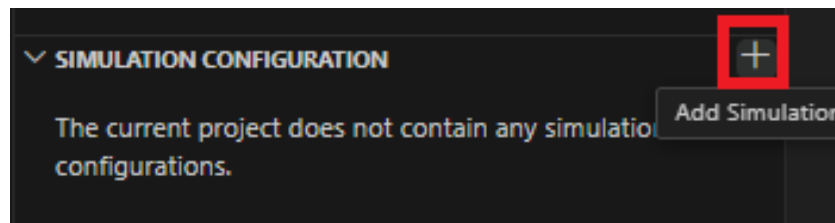


Figure 19. Add a simulation configuration.

Fill and save the configuration tab with the name **lab1_sim**, and the next options:

```

-top work.test -uvm 1.2 -L work +acc+b -waves
lab1_test.mxd -dump-agg -sv_seed random -timescale
1ns/1ps +UVM_TIMEOUT=10000 +UVM_VERBOSITY=UVM_MEDIUM
+UVM_NO_DEPRECATED +UVM_OBJECT_MUST_HAVE_CONSTRUCTOR

```

-uvm 1.2 switch enables DSIM to look for the **uvm_pkg** in the installation directory. The **work.test** is the program that you just created in the **test.sv** file.

Recompile, as shown in step 14 from "*DSIM first steps*".

After the successful compilation of your design, run the test.

5. Simulate this UVM testbench by clicking the 'Run' button beside **lab1_sim** in the **SIMULATION CONFIGURATION** from the DSIM Desktop.

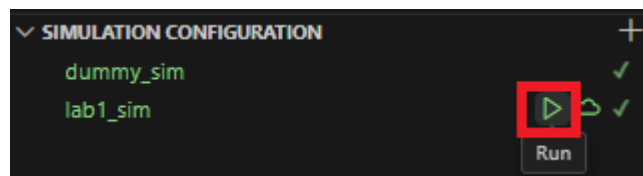
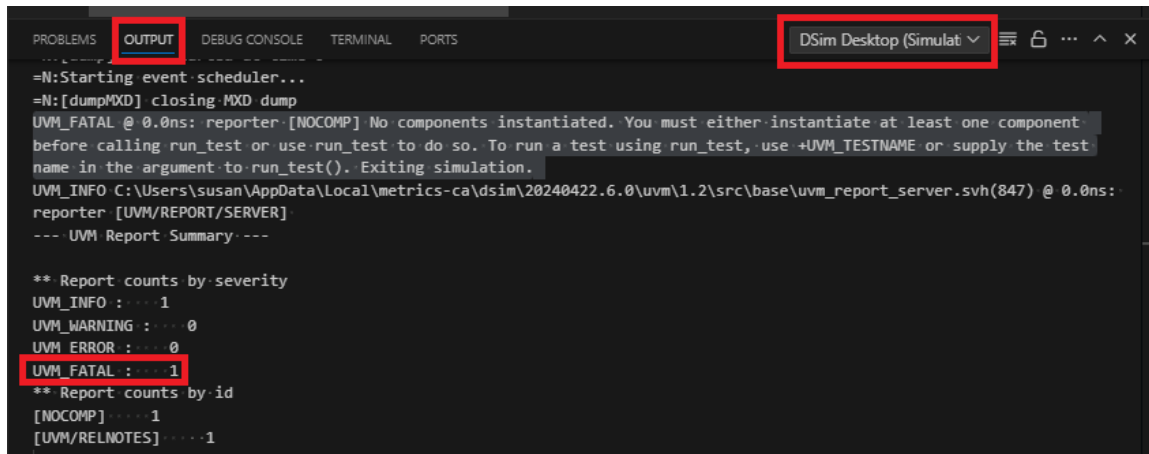


Figure 20. Running lab1 test.

The output is in the **dsim.log** file that can be found in the main directory of the project. It is also possible to check the log in the Output tab of the VSC, just be sure to choose "*DSIM Desktop (Simulation)*".

You should see the following fatal error:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
DSim Desktop (Simulation)
=N:Starting event scheduler...
=N:[dumpMXD] closing MXD dump
UVM_FATAL @ 0.0ns: reporter:[NOCOMP] No components instantiated. You must either instantiate at least one component
before calling run_test or use run_test to do so. To run a test using run_test, use +UVM_TESTNAME or supply the test
name in the argument to run_test(). Exiting simulation.
UVM_INFO C:\Users\susan\AppData\Local\metrics-ca\dsim\20240422.6.0\vvm\1.2\src\base\vvm_report_server.svh(847) @ 0.0ns:
reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : ... 1
UVM_WARNING : ... 0
UVM_ERROR : ... 0
UVM_FATAL : ... 1
** Report counts by id
[NOCOMP] : ... 1
[UVM/RELNOTES] : ... 1
```

Figure 21. Fatal error.

There are two reasons for this fatal error:

First, you do not have any UVM test class.

Second, even if you had a UVM test class, you did not specify the test to be executed via the **+UVM_TESTNAME** switch

You will correct this by creating the test class first.

6. Open the **test_collection.sv** file. Enter the following code:

```
class test_base extends uvm_test;
    `uvm_component_utils(test_base)
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
endclass
```

Some explanations are needed for this code.

The test class (**test_base**) is created by extending **uvm_test** base class.

```
class test_base extends uvm_test;
    ...
endclass
```

As was mentioned earlier, there are three key elements of the UVM package that you must understand and apply them to your UVM testbench.

One of them is the UVM factory.

Every UVM class that you create must be registered into this UVM factory. The UVM package provides a macro to simplify this process. The macro for a component class is: ``uvm_object_utils`. The name that you enter into the argument of the macro must match the class name:

You will see the effects and benefits of registering user classes into the UVM factory in the next lab (lab2). For now, consider this as a requirement.

```
class test_base extends uvm_test;
    `uvm_component_utils(test_base)
```

Once you register a class into the UVM factory, you will use the UVM factory to construct UVM testbench objects. The UVM factory requires that you define the constructor **new()** for component classes in a very specific way:

```
function new(string name, uvm_component parent);
    super.new(name, parent);
endfunction
```

There must only be two arguments to the constructor - a **string** field for the instance name of the object and a **uvm_component** field for the object's structural parent. These two fields must be passed on the base class via the **super.new()** call. Again, this is a requirement that you must obey in order to use the UVM factory.

7. Save and close the file.

Once you have the test class defined, you will need to include this file to be compiled as part of your **test.sv** code. (A better alternative is to embed the code inside a SystemVerilog package, then import the package into the program file. This will be done

for you in subsequent labs.)

8. Open the **test.sv** file in an editor.
9. Add the following include statements:

```
program automatic test;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  `include "labs\lab1\test_collection.sv"

  initial begin
    ...
  end
.
```

Compile and simulate this code again. This time, use the **+UVM_TESTNAME** switch to specify the test for the UVM execution manager (one of the three key elements of the UVM package) to run.

10. Hover over the lab1_sim from the **SIMULATION CONFIGURATION** click right click it, choose "Configure Simulation".

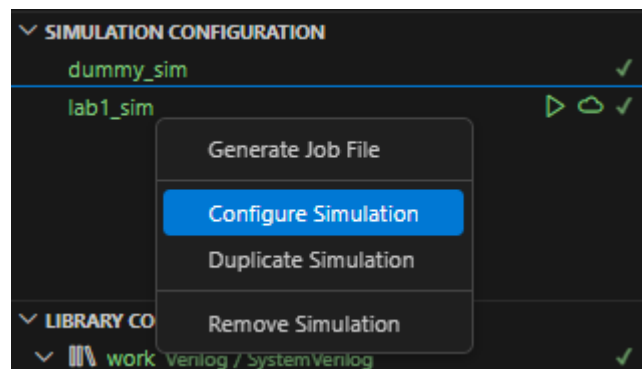


Figure 22. Reconfiguring menu for lab1_sim.

Add the **+UVM_TESTNAME=test_base** plusarg to the options and save the changes.

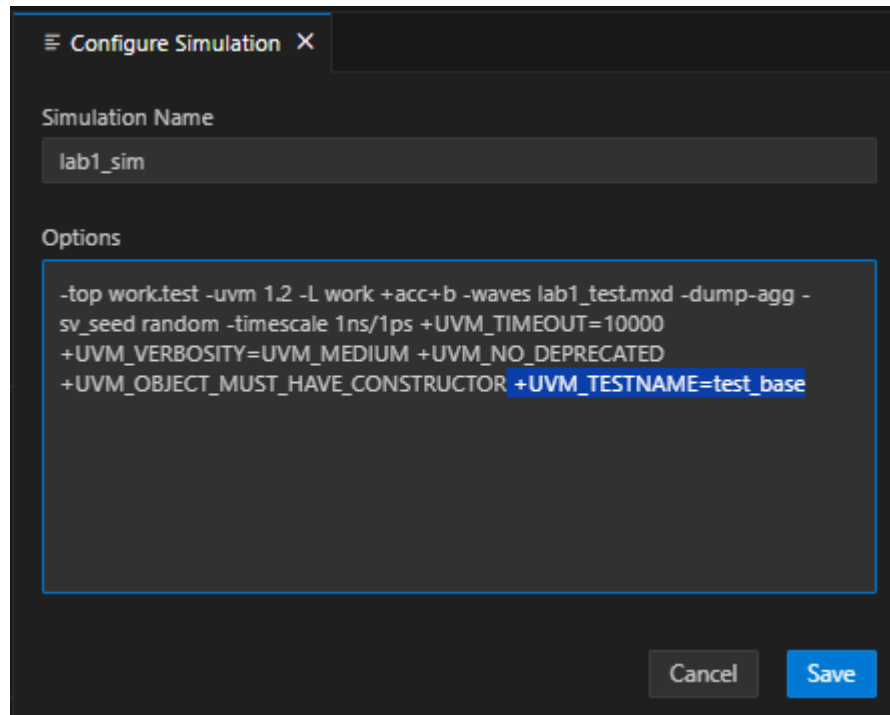


Figure 23. Adding the **+UVM_TESTNAME** plusarg to the simulation.

Recompile and then run the **lab1_sim** through the DSIM desktop.

At the end of simulation, you should see something like the following:

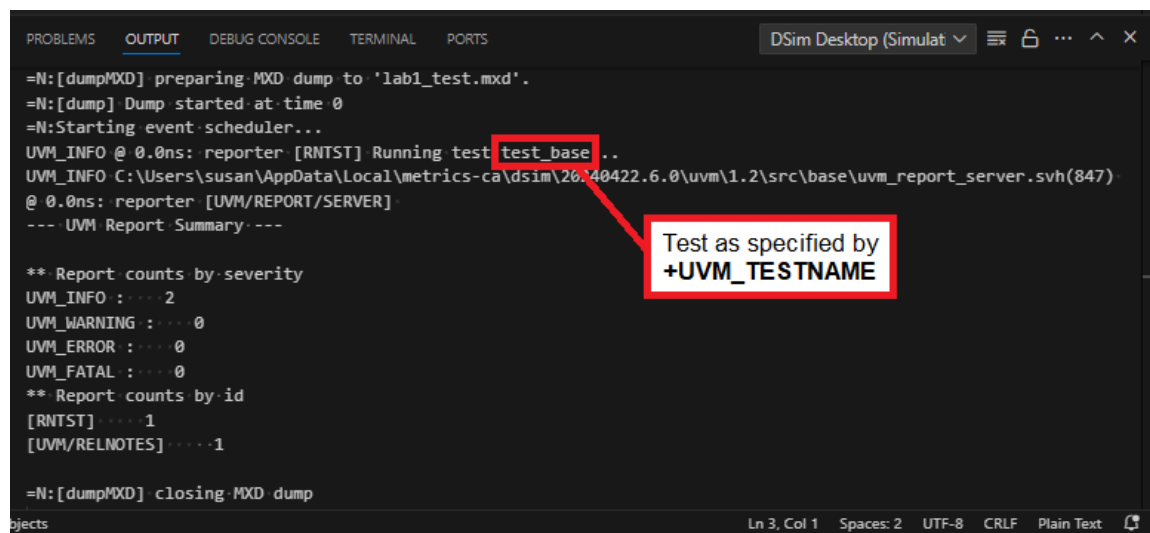


Figure 24. Output when adding **+UVM_TESTNAME** plusarg.

Before proceeding, let's make sure you understand what's happening here.

- Question 1.** Which one of the three key UVM elements of the UVM package is responsible for reading the **+UVM_TESTNAME** switch?

Question 2. What code in the **test.sv** file caused this key element to run and look at the **+UVM_TESTNAME** switch?

Question 3. How does this key element make use of the test class name provided via the **+UVM_TESTNAME** switch?

The object in UVM that reads the **+UVM_TESTNAME** switch is the UVM execution manager (**uvm_root**) which you started via **run_test()**. It makes use of the UVM factory to create the top test object from the class name provided.

This top test is called **uvm_test_top**. The **uvm_test_top** object sits at the top of the entire UVM test hierarchy. It is the root parent of all your UVM test components.

Since the UVM execution manager is the creator of the test object, it is also aware of the entire UVM component structural hierarchy from the test on down.

The UVM execution manager is a singleton object of the **uvm_root** class. You can retrieve the handle to this manager by calling **uvm_root::get()**. You can use this handle to print the test structural topology.

11. Open the **test_collection.sv** file in an editor.
12. Add the following start of simulation phase method to the **test_base** class:

```
class test_base extends uvm_test;
  `uvm_component_utils(test_base)
  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction
  virtual function void
  start_of_simulation_phase(uvm_phase phase);
    super.start_of_simulation_phase(phase);
    uvm_root::get().print_topology;
  endfunction
```

13. Recompile and then run the **lab1_sim** through the DSIM desktop.

At the end of simulation, you should now see the test topology printed:

```

UVM_INFO C:\Users\susan\AppData\Local\metrics-ca\dsim\20240422.6.0\vvm\1.2\src\base\vvm_root.svh(579) @ 0.0ns: reporter [UVMTOP] UVM testbench topology:
-----
Name.....Type.....Size..Value
-----
uvm_test_top..test_base.....|@334
-----

```

Figure 25. Printing test topology.

At this point you only have the **test_base** class object called **uvm_test_top**. So, that's all you see.

As your test bench becomes more complex, you will find this set of information to be very helpful during debugging sessions.

It is also useful for debugging to see all the user classes registered in the UVM factory. You can get this information by doing the following steps.

14. Open the **test_collection.sv** file in an editor.
15. Add a statement to print factory registry:

```

virtual function void start_of_simulation_phase(uvm_phase
phase);
    super.start_of_simulation_phase
    uvm_root::get().print_topology();
    uvm_factory::get().print();
endprogram

```

16. Recompile and then run the **lab1_sim** through the DSIM desktop.

You should now see the factory registry content:

```

UVM_INFO C:\Users\susan\AppData\Local\metrics-ca\dsim\20240422.6.0\vvm\1.2\src\base\vvm_factory.svh
(1645) @ 0.0ns: reporter [UVM/FACTORY/PRINT]
#### Factory Configuration (*)
- No instance or type overrides are registered with this factory
All types registered with the factory: 51 total
Type Name
-----
test_base
(*) Types with no associated type name will be printed as <unknown>
####

```

Figure 26. Printing factory classes.

Task 2. Debugging essentials

One common problem encountered when running a test is hung simulation. Simulation starts then hangs at some point during simulation. The cause is usually due to a zero-time infinite loop or a broken (unconnected) communication link or a dead locked code.

To debug this, it is helpful to find out at what point did the simulation hang and how did the simulation get to that point. You can embed a trace statement as the first executable statement (after calling the super method) of a method to enable this.

1. Open the **test_collection.sv** file with an editor.
2. Add the following UVM report messages:

```
class test_base extends uvm_test;
  `uvm_component_utils(test_base)
  function new(string name, uvm_component parent);
    super.new(name, parent);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
  endfunction
  virtual function void
  start_of_simulation_phase(uvm_phase phase);
    super.start_of_simulation_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    uvm_root::get().print_topology;
    uvm_factory::get().print();
  endfunction
```

In the UVM report message macro, the first argument is the **ID** of the message. Choose it carefully by picking a brief name that is likely to be unique. It will be very useful when you need to control the report messages to display. In this code, we want to clearly show that the message is intended for tracing the execution of all user methods. Thus, the **ID** is set to **TRACE**.

It is helpful in debugging to know the method name and the file context. The second argument, **\$sformatf("%m")**, will give you this information.

The third argument of the report message macro is the verbosity of the message. By setting it to **UVM_HIGH**, this message will be filtered out in normal execution (only **UVM_NONE**, **UVM_LOW** and **UVM_MEDIUM** messages are displayed by default). But you will be able to enable the message via a run-time switch during debugging sessions.

3. Recompile and then run the **lab1_sim** through the DSIM desktop.

You should see the same display as before. None of the added **TRACE** report messages are displayed.

4. Hover over the lab1_sim from the **SIMULATION CONFIGURATION** click right click it, choose "Configure Simulation".

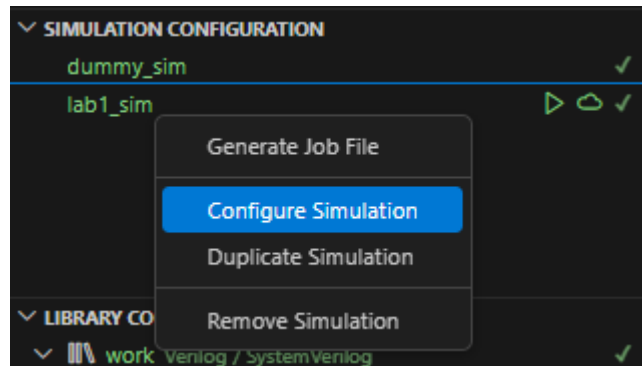


Figure 27. Reconfiguring menu for **lab1_sim**.

Modify the **+UVM_VERBOSITY** from **UVM_MEDIUM** to **UVM_HIGH** and save the changes.

This change sets the run-time switch **+UVM_VERBOSITY=UVM_HIGH**.

Recompile and then run the **lab1_sim** through the DSIM desktop.

Do you see the messages? Hard to read?

5. Try this.

Open a new WSL terminal tab.

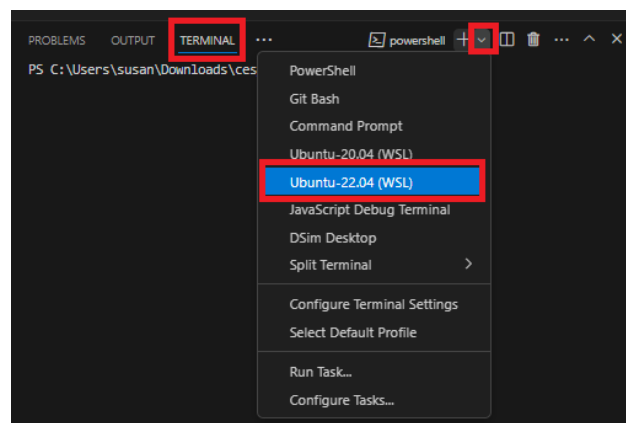


Figure 28. New WSL tab.

Go into **ces_uvm-1.2_2021.09** directory, use the next command:

```
> grep TRACE dsim.log
```

Now, you can clearly see that the only thing that happened during this test run was that the **test_base** instance **uvm_test_top** was constructed and its **start_of_simulation_phase** method was executed afterwards.

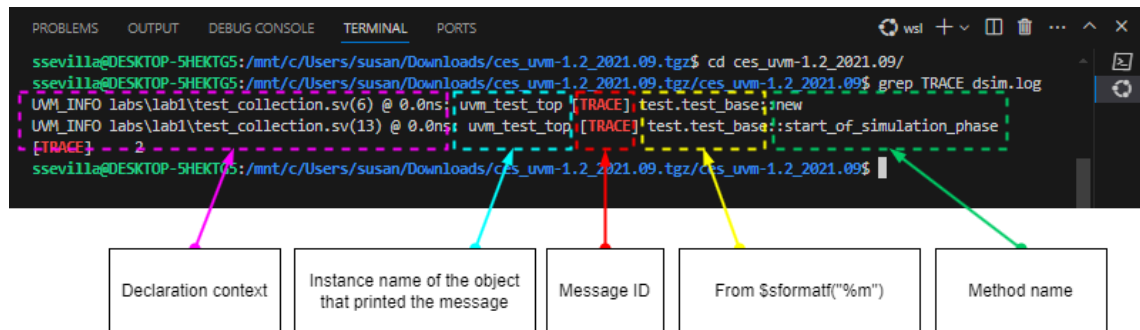


Figure 29. Using grep for tracking **TRACE**.

There are other ways you can make use of the **ID** field.

You may want to turn on **TRACE** report messages without affecting other report messages. This can be done via the **+uvm_set_verbosity** switch.

6. Hover over the lab1_sim from the **SIMULATION CONFIGURATION** click right click it, choose "Configure Simulation".

Modify the **+UVM_VERBOSITY** from **UVM_HIGH** to **UVM_DEBUG** and save the changes.

Recompile and then run the **lab1_sim** through the DSIM desktop.

Rename the **dsim.log** file as **dsim_debug.log**

Look at the simulation result in the **dsim_debug.log** file. You will see that every report message is captured. This, in a typical debugging session, will overwhelm you with too much data. Debugging then becomes very difficult.

You can use the **+uvm_set_verbosity** switch to apply the verbosity to only the area of debugging that you are interested in.



DSim does not support the +uvm_set_verbosity switch, it ignores the command. So steps 7 to 12 can be avoided. Feel free to just read them and learn about this switch.

7. Hover over the lab1_sim from the **SIMULATION CONFIGURATION** click right click it, choose "Configure Simulation".

Modify the **+UVM_VERBOSITY** from **UVM_DEBUG** to **UVM_MEDIUM**.

Add **+uvm_set_verbosity=*,TRACE,UVM_DEBUG,time,0** and save the changes.

Recompile and then run the **lab1_sim** through the DSIM desktop.

8. Compare the two simulation results. In the WSL terminal use the next command:

```
> tkdiff dsim.log dsim_debug.log
```

By using the **uvm_set_verbosity** switch and selecting the **ID** to display, you can control and manage the UVM report message without being overwhelmed.

Recall these statements in the **start_of_simulation_phase** method:

```
uvm_root::get().print_topology;  
uvm_factory::get().print();
```

You may want to control these two print statements also. Unfortunately, these print statements are not easily filtered out.

What you can do, though, is to wrap the code you want to control around a user created report message.

9. Open the **test_collection.sv** file with an editor.
10. Wrap the print statements as user defined UVM report messages:

```
class test_base extends uvm_test;  
  `uvm_component_utils(test_base)  
  function new(string name, uvm_component parent);  
    super.new(name, parent);  
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);  
  endfunction  
  virtual function void  
  start_of_simulation_phase(uvm_phase phase);  
    super.start_of_simulation_phase(phase);  
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);  
    if (uvm_report_enabled(UVM_MEDIUM, UVM_INFO,  
"TOPOLOGY")) begin  
      uvm_root::get().print_topology;  
    end  
end
```

```
    if (uvm_report_enabled(UVM_MEDIUM, UVM_INFO,
"FACTORY")) begin
        uvm_factory::get().print();
    end
endfunction
```

11. Recompile and then run the **lab1_sim** through the DSIM desktop.

Again, you should see the same display as before. The topology and the factory registry are printed. The **TRACE** report messages are not displayed.

12. Hover over the lab1_sim from the **SIMULATION CONFIGURATION** click right click it, choose "*Configure Simulation*".

Modify the **+UVM_VERBOSITY** from **UVM_MEDIUM** to **UVM_HIGH**.

Add **+uvm_set_verbosity=*, TOPOLOGY, UVM_NONE, time, 0** and save the changes.

Recompile and then run the **lab1_sim** through the DSIM desktop.

Now, the **TOPOLOGY** message is filtered out, but you see everything else at the **UVM_HIGH** verbosity.

The impact of filtering will be much more obvious when you deal with a real testbench.

The steps 7 to 12 are described for the way that the switch should be used DSIm, but since this is not possible with this tool, for illustrating why this switch is useful, some simulations were done in other simulation tools for showing the expected output. Also we dig a little deeper about how to use this switch.

Digging a little deeper in the +uvm_set_verbosity

The use of the **+uvm_set_verbosity** switch can be divided into two main ways: per phase and per time. This allows the user to specify the phase or the time to modify the verbosity of all the test or just a specific component.

Per phase	Command: +uvm_set_verbosity=<comp>,<id>,<verbosity>,<phase>
	Example: +uvm_set_verbosity=uvm_test_top.m_env,_ALL_,UVM_HIGH,run Just for the run phase, modify the verbosity to UVM_HIGH of all (_ALL_ means all IDS) the messages in the m_env component.
Per time	Command: +uvm_set_verbosity=<comp>,<id>,<verbosity>,time,<time>
	Example: +uvm_set_verbosity=*,TRACE,UVM_DEBUG,time,0 At time zero, modify the verbosity to UVM_DEBUG of all the messages that have TRACE as ID in every component of the test.

Command: +uvm_set_verbosity=*,TRACE,<verbosity>,time,0

UVM_VERBOSITY =		UVM_NONE					UVM_LOW					UVM_MEDIUM					UVM_HIGH					UVM_FULL					UVM_DEBUG				
<verbosity>	in code uvm_info ID	NONE	LOW	MEDIUM	HIGH	FULL	DEBUG	NONE	LOW	MEDIUM	HIGH	FULL	DEBUG	NONE	LOW	MEDIUM	HIGH	FULL	DEBUG	NONE	LOW	MEDIUM	HIGH	FULL	DEBUG	NONE	LOW	MEDIUM	HIGH	FULL	DEBUG
		NONE	LOW	MEDIUM	HIGH	FULL	DEBUG	NONE	LOW	MEDIUM	HIGH	FULL	DEBUG	NONE	LOW	MEDIUM	HIGH	FULL	DEBUG	NONE	LOW	MEDIUM	HIGH	FULL	DEBUG	NONE	LOW	MEDIUM	HIGH	FULL	DEBUG
UVM_NONE	TRACE	✓	X	X	X	X	X	✓	X	X	X	X	X	✓	X	X	X	X	X	✓	X	X	X	X	X	✓	X	X	X	X	X
	OTHER IDS	✓	X	X	X	X	X	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
UVM_LOW	TRACE	✓	✓	X	X	X	X	✓	✓	X	X	X	X	✓	✓	X	X	X	X	✓	✓	X	X	X	X	✓	✓	X	X	X	X
	OTHER IDS	✓	X	X	X	X	X	✓	✓	X	X	X	X	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
UVM_MEDIUM	TRACE	✓	✓	✓	X	X	X	✓	✓	✓	✓	X	X	✓	✓	✓	X	X	X	✓	✓	✓	✓	X	X	✓	✓	✓	X	X	X
	OTHER IDS	✓	X	X	X	X	X	✓	✓	X	X	X	X	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
UVM_HIGH	TRACE	✓	✓	✓	✓	X	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	X	X
	OTHER IDS	✓	X	X	X	X	X	✓	✓	X	X	X	X	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
UVM_FULL	TRACE	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X	✓	✓	✓	✓	✓	X
	OTHER IDS	✓	X	X	X	X	X	✓	✓	X	X	X	X	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
UVM_DEBUG	TRACE	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
	OTHER IDS	✓	X	X	X	X	X	✓	✓	X	X	X	X	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Figura 30. UVM_VERBOSITY and uvm_set_verbosity table.

As shown in figure 30, if a test is simulated with **+UVM_VEBOSITY=UVM_MEDIUM** and **+uvm_set_verbosity=*,TRACE,UVM_DEBUG,time,0** switches (as is expected in the previous task) it is expected that in the log every message with a **MEDIUM** or lower verbosity appears in the log, but also every message from the test with **TRACE** as **ID** and with a **uvm_info** verbosity equal or lower than **UVM_DEBUG** is expected to appear in the log.

As an example, we added some messages to the new function of the test_base class, with different IDs and verbosity. So that we have the next function:

```
function new(string name, uvm_component parent);
    super.new(name, parent);
    `uvm_info("TRACE", $sformatf("NONE %m"), UVM_NONE);
    `uvm_info("TRACE", $sformatf("LOW %m"), UVM_LOW);
    `uvm_info("TRACE", $sformatf("MEIDUM %m"), UVM_MEDIUM);
    `uvm_info("TRACE", $sformatf("HIGH %m"), UVM_HIGH);
    `uvm_info("TRACE", $sformatf("FULL %m"), UVM_FULL);
    `uvm_info("TRACE", $sformatf("DEBUG %m"), UVM_DEBUG);
    `uvm_info("Aid", $sformatf("NONE %m"), UVM_NONE);
    `uvm_info("Bid", $sformatf("LOW %m"), UVM_LOW);
    `uvm_info("Cid", $sformatf("MEIDUM %m"), UVM_MEDIUM);
    `uvm_info("Did", $sformatf("HIGH %m"), UVM_HIGH);
    `uvm_info("Eid", $sformatf("FULL %m"), UVM_FULL);
    `uvm_info("Fid", $sformatf("DEBUG %m"), UVM_DEBUG);
endfunction
```

After running the test, in another simulation tool, with **+UVM_VEBOSITY=UVM_MEDIUM** and **+uvm_set_verbosity=*,TRACE,UVM_DEBUG,time,0** switches, the next output was obtained:

```
UVM_INFO labs\lab1\test_collection.sv(9) @ 0.0ns: uvm_test_top [TRACE] NONE test.test_base.new
UVM_INFO labs\lab1\test_collection.sv(10) @ 0.0ns: uvm_test_top [TRACE] LOW test.test_base.new
UVM_INFO labs\lab1\test_collection.sv(11) @ 0.0ns: uvm_test_top [TRACE] MEIDUM test.test_base.new
UVM_INFO labs\lab1\test_collection.sv(12) @ 0.0ns: uvm_test_top [TRACE] HIGH test.test_base.new
UVM_INFO labs\lab1\test_collection.sv(13) @ 0.0ns: uvm_test_top [TRACE] FULL test.test_base.new
UVM_INFO labs\lab1\test_collection.sv(14) @ 0.0ns: uvm_test_top [TRACE] DEBUG test.test_base.new
UVM_INFO labs\lab1\test_collection.sv(15) @ 0.0ns: uvm_test_top [Aid] NONE test.test_base.new
UVM_INFO labs\lab1\test_collection.sv(16) @ 0.0ns: uvm_test_top [Bid] LOW test.test_base.new
UVM_INFO labs\lab1\test_collection.sv(17) @ 0.0ns: uvm_test_top [Cid] MEIDUM test.test_base.new
```

Figure 31. Filtering messages with **+UVM_VEBOSITY=UVM_MEDIUM** and **+uvm_set_verbosity=*,TRACE,UVM_DEBUG,time,0** switches.

Task 3. Add in an Environment

An environment with an agent which contains a sequencer and driver needs to be created. These classes are the same as shown in the lecture.

1. Bring in the environment and transaction files, open a WSL terminal and run the next command:

> make environment

Add these to the **test.sv** file.

2. Open the **test.sv** file with an editor.
3. Add the following include statements (must be in the order show)

```
program automatic test;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  `include "labs\lab1\packet.sv"
  `include "labs\lab1\driver.sv"
  `include "labs\lab1\input_agent.sv"
  `include "labs\lab1\router_env.sv"
  `include "labs\lab1\test_collection.sv"
```

4. Open the **test_collection.sv** file with an editor.
5. Instantiate and construct an environment object in the test class:

```

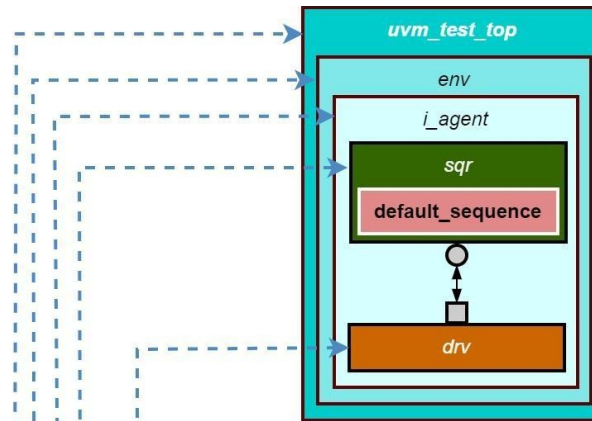
class test_base extends uvm_test;
  `uvm_component_utils(test_base)
  router_env env;
  ...
  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    env = router_env::type_id::create("env", this);
  endfunction
  ...
endclass

```

Use factory to create object

6. Be sure to have the **+UVM_VERBOSITY** switch with **UVM_MEDIUM** in the configuration of the test. Recompile and then run the **lab1_sim** through the DSIM desktop.

You should see a fuller topology and the factory content. (You will see **recording_details** in addition to what's shown below)



```

UVM_INFO C:\Users\susan\AppData\Local\metrics-ca\dsim\20240422.6.0\uvm\1.
2\src\base\uvm_root.svh(579) @ 0.0ns: reporter [UVMTOP] UVM testbench topology:
-----
Name                                     Type                                     Size  Value
-----
uvm_test_top                           test_base                               @334
  env                                   router_env                             @353
    i_agt                               input_agent                             @366
      drv                               driver                                 @517
        rsp_port                        uvm_analysis_port                       @536
        seq_item_port                  uvm_seq_item_pull_port                 @526
      sqr                               uvm_sequencer                           @380
        rsp_export                      uvm_analysis_export                     @389
        seq_item_export                uvm_seq_item_pull_imp                   @507
        arbitration_queue              array                                  0
        lock_queue                     array                                  0
        num_last_reqs                   integral                                32    'd1
        num_last_rsps                   integral                                32    'd1
-----

UVM_INFO C:\Users\susan\AppData\Local\metrics-ca\dsim\20240422.6.0\uvm\1.
2\src\base\uvm_factory.svh(1645) @ 0.0ns: reporter [UVM/FACTORY/PRINT]
#### Factory Configuration (*)

  No instance or type overrides are registered with this factory

All types registered with the factory: 56 total
  Type Name
  -----
  driver
  input_agent
  packet
  router_env
  test_base
(*) Types with no associated type name will be printed as <unknown>

```

Figure 32. Printing topology.



DSim does not support the `+uvm_set_verbosity` switch, it ignores the command. So step 7 can be avoided.

7. Open the configuration of the `lab1_sim`, modify the `+UVM_VEBOSITY` to `UVM_HIGH` and

add the next switch:

```
+uvm_set_verbosity=\*,TRACE,UVM_DEBUG,time,0
```

You can begin to see the impact of the UVM report message filtering.

UVM provides you with a great deal of control over what report messages to generate. Choose the report message IDs carefully. They can become very handy during debugging.

Task 4. Creating User Packages

(The following is information only. No action is required.)

To enable better management of class names and compiled binaries, it is highly useful to include these class codes inside SystemVerilog packages.

For the subsequent labs, the files you have been working with will be moved inside packages. This will be done for you.

Be aware that package coding guidelines vary from company to company and project to project. It is highly recommended that once you return back to your home environment, consult your project manager and include your class code inside packages as directed by the project and company coding guidelines.

You are done with lab 1!

Solutions

Test_collection.sv // Avoiding +uvm_set_verbosity steps

```

class test_base extends uvm_test;
  `uvm_component_utils(test_base)
  router_env env;

  function new(string name, uvm_component parent);
    super.new(name, parent);
    `uvm_info("TRACE", $sformatf("HIGH %m"), UVM_HIGH);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    `uvm_info("TRACE", $sformatf("%m"), UVM_HIGH);
    env = router_env::type_id::create("env", this);
  endfunction

  virtual function void
start_of_simulation_phase(uvm_phase phase);
    super.start_of_simulation_phase(phase);
    uvm_root::get().print_topology;
    uvm_factory::get().print();
    `uvm_info("TRACE", $sformatf("HIGH %m"), UVM_HIGH);;
  endfunction
endclass

```

test.sv

```
program automatic test;
  import uvm_pkg::*;
  `include "uvm_macros.svh"
  `include "labs\lab1\packet.sv"
  `include "labs\lab1\driver.sv"
  `include "labs\lab1\input_agent.sv"
  `include "labs\lab1\router_env.sv"
  `include "labs\lab1\test_collection.sv"

  initial begin
    $timeformat(-9, 1, "ns", 10); // Format Verilog time
    run_test();
  end

endprogram
```